МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Львівський національний університет імені Івана Франка

Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи №3

«Ітератори, генератори та потокова обробка даних»

з дисципліни «Професійний Python»

Варіант №5 — «Потокова обробка каталогу автомобілів»

Виконав:

студент групи ФЕП-32

Горин Михайло

Репозиторій: github.com/MykhailoG212/Prof_python

Львів — 2026

2. Номер і тема лабораторної роботи

Лабораторна робота №3. Тема: «Ітератори, генератори та потокова обробка даних».

3. Мета роботи

Сформувати практичні навички роботи з ітераційною моделлю Python: iterable та iterator, протоколом `__iter__()` / `__next__()`, generator functions і `yield`, generator expressions, конструкцією `yield from`, модулем `itertools`, принципом lazy evaluation та потоковою обробкою великих наборів даних. Побудувати lazy pipeline, у якому записи проходять послідовно через кілька етапів без повної матеріалізації проміжних колекцій, та порівняти eager і lazy обробку за часом і пікової пам'яті.

4. Номер індивідуального варіанта

Варіант №5 — «Потокова обробка каталогу автомобілів». Робота є продовженням ЛР1 і ЛР2: код розміщено в підпакеті `car_catalog.streaming` того самого проєкту, репозиторій https://github.com/MykhailoG212/Prof_python.

5. Постановка задачі

Необхідно розробити потоковий конвеєр обробки великого набору даних про автомобілі. Вхідний файл — CSV зі схемою `id,brand,model,year,price,mileage`; інформація має проходити послідовно через кілька lazy-етапів (читання, очищення, розбір, перевірка, фільтрація, перетворення, агрегація), а повний файл не повинен завантажуватися в пам'ять.

5.1. Вимоги варіанта №5

Вимога варіанта	Реалізація
streaming validation автомобільних даних	`validate_car_rows()` + `ValidationStats`
фільтрація за маркою та роком	`filter_cars(cars, brand, year)`
найдорожчий автомобіль	`calculate_statistics().most_expensive`
streaming average price	`streaming_average()`, `calculate_statistics().average_price`
автомобіль із найменшим пробігом	`calculate_statistics().lowest_mileage`
batching	`batched()`, `flatten_batches()`

Вимога варіанта	Реалізація
перші N записів за умовою	`first_matching_cars()` (`islice`)
groupby за маркою після сортування	`group_cars_by_brand()` (`sorted` + `groupby`)

5.2. Вимоги до конвеєра

Згідно з розділом 40 завдання, конвеєр повинен містити такі елементи:

№	Вимога	Де виконано
1	джерело даних	`data/cars.csv`, синтетичні CSV з `write_synthetic_dataset()`
2	generator для читання	`read_lines()`, `clean_lines()`
3	generator для очищення	`clean_lines()`
4	generator для parsing	`parse_csv_rows()`
5	generator для filtering	`filter_cars()`, `first_matching_cars()`
6	generator для transformation	`summarize_cars()`, `format_car_rows()`
7	aggregation stage	`calculate_statistics()`, `streaming_average()`
8	generator expression	`summarize_cars()`, `(car.price for car in cars)`
9	≥ 3 функції з `yield`	понад десять (розділ 10)
10	конструкція `yield from`	`read_lines()`, `parse_csv_rows()`, `flatten_nested()`
11	≥ 3 засоби `itertools`	`chain`, `islice`, `accumulate`, `pairwise`, `groupby`, `count`
12	використання `islice`	`first_matching_cars()`, `batched()`, `first_car_ids()`
13	streaming processing	розділи 14–16
14	batch processing	`batched()`
15	обробка ≥ 100 000 records	експеримент на 10 000, 100 000 і 500 000 записів
16	порівняння eager і lazy	розділи 22–24
17	оцінка memory consumption	`tracemalloc`, таблиця 2
18	аналіз переваг і недоліків lazy	розділ 26

6. Опис input dataset

Вхідні дані — CSV-файл із заголовком `id,brand,model,year,price,mileage`. Кожен рядок описує один

автомобіль; значення типізуються під час валідації (`int`, `str`, `str`, `int`, `float`, `int`). Запис вважається коректним, якщо виконуються правила:

- `id` — ціле додатне число;
- `brand` і `model` — непорожні рядки;
- `year` — ціле число, не менше 1886;
- `price` — скінченне невід'ємне число (`NaN` та `inf` відхиляються);
- `mileage` — ціле невід'ємне число; відсутні та зайві поля також вважаються помилкою.

Навчальний файл `data/cars.csv` містить сім рядків, серед яких один навмисно некоректний:

```
id,brand,model,year,price,mileage
1,Tesla,Model Y,2023,50000,12000
2,Toyota,Camry,2020,25000,45000
3,Honda,Civic,2021,22000,30000
4,Toyota,Corolla,2019,18000,60000
5,BMW,M3,2022,72000,24000
invalid,Unknown,Model,2020,not-a-price,100
6,Porsche,911 Carrera,2023,132000,9200
```

Для експерименту `write_synthetic_dataset(path, size, invalid_every)` створює відтворюваний CSV заданого розміру: марки `Brand_00`…`Brand_49`, моделі `Model_<index>`, роки 2000–2024, ціни від 10 000 до 99 999 та пробіг, обчислений за індексом. Після кожних `invalid_every` валідних рядків додається контрольний некоректний запис, тому кількість відхилених рядків наперед відома.

7. Структура проєкту

Потокову обробку винесено в окремий підпакет `car_catalog.streaming`, що не змінює модулі ЛР1 і ЛР2. Рекомендовану в завданні структуру `stream_processor` адаптовано під проєкт: пакет має назву `car_catalog`, а етапи конвеєра розділено на модулі за відповідальністю.

```
lab2code/
├── data/
│   └── cars.csv                # навчальний вхідний CSV
├── src/
│   └── car_catalog/
│       ├── models.py, services.py, processors.py, analytics.py, ...   # ЛР1–ЛР2
│       ├── main.py             # меню; пункти 11–12 запускають демо й benchmark ЛР3
│       └── streaming/
│           ├── __init__.py     # публічний інтерфейс підпакета
│           ├── __main__.py     # python -m car_catalog.streaming
│           ├── models.py       # CarRecord, CarSummary, ValidationStats, CarStatistics
│           ├── iterators.py    # власний iterator, нескінченний generator, yield from
│           ├── readers.py      # read_lines, clean_lines
│           ├── parsers.py      # parse_csv_rows
│           ├── validation.py   # validate_car_rows
│           ├── filters.py      # filter_cars, first_matching_cars, find_car_by_id
│           ├── transformations.py# summarize_cars, format_car_rows
│           ├── batches.py      # batched, flatten_batches
```

```
│           ├── analytics.py      # агрегація та itertools
│           ├── pipeline.py       # збирання конвеєра
│           ├── benchmark.py      # eager vs lazy, time, peak memory
│           └── demo.py           # демонстраційний запуск
├── tests/
│   └── test_streaming.py         # тести ЛР3 (поряд із тестами ЛР1 та ЛР2)
├── pyproject.toml
└── README.md
```

Модуль	Призначення
`models.py`	`CarRecord` і `CarSummary` (`NamedTuple`), лічильник якості даних `ValidationStats`, результат агрегації `CarStatistics`
`iterators.py`	власний iterator `CarIdIterator`, нескінченний `infinite_car_ids()` з `islice`, рекурсивний `flatten_nested()`
`readers.py`	джерело даних: построкове читання файлу та очищення рядків
`parsers.py`	розбір CSV-рядків у словники
`validation.py`	типізація та перевірка записів, облік причин відхилення
`filters.py`	lazy-фільтрація, перші N збігів, пошук за ID з достроковим завершенням
`transformations.py`	перетворення записів у короткі представлення та CSV-рядки
`batches.py`	розбиття потоку на пакети та зворотне «розгортання»
`analytics.py`	агрегація за один прохід; `accumulate`, `pairwise`, `groupby`
`pipeline.py`	з'єднання етапів, підтримка кількох файлів через `chain`
`benchmark.py`	генерація даних, eager і lazy обробка, вимірювання часу та пам'яті

8. Пояснення iterable та iterator

Iterable — об'єкт, з якого можна отримати iterator (список, кортеж, словник, файл). **Iterator** — об'єкт, що зберігає поточний стан обходу, має метод `__next__()`, повертає наступний елемент і після завершення піднімає `StopIteration`; його метод `__iter__()` повертає сам об'єкт, тому `iter(iterator) is iterator`. Iterator зазвичай є одноразовим (one-shot): другий прохід по вичерпаному iterator нічого не повертає.

Цикл `for` концептуально виконує `iterator = iter(values)`, потім у циклі викликає `next(iterator)` і завершується при `StopIteration`. Саме цей протокол використовують `sum()`, `min()`, `max()`, `sorted()`, `list()` та comprehensions. У проєкті його повністю реалізує `CarIdIterator`, а файловий об'єкт у `read_lines()` є iterable, що видає рядки по одному.

Поняття	Що зберігає / робить	Приклад у проєкті
iterable	вміє створити iterator (`__iter__`)	`list[CarRecord]`, файловий об'єкт, `Path.open()`
iterator	стан обходу + `__next__`, одноразовий	`CarIdIterator`, результат `iter(list)`
generator function	функція з `yield`; повертає generator object	`read_lines()`, `filter_cars()`
generator expression	компактний lazy-вираз `(x for x in ...)`	`summarize_cars()`

9. Код власного iterator

Клас `CarIdIterator` реалізує iterator protocol вручну: `__iter__()` повертає `self`, а `__next__()` повертає наступний ID або піднімає `StopIteration`, коли досягнуто межі `stop`. Конструктор перевіряє, що початковий ID додатний, а кінцевий не менший за початковий.

```
class CarIdIterator(Iterator[int]):
    """Власний одноразовий iterator послідовних додатних ID автомобілів."""


    def __init__(self, start: int = 1, stop: int | None = None) -> None:
        if start < 1:
            raise ValueError("Початковий ID має бути додатним.")
        if stop is not None and stop < start:
            raise ValueError("Кінцевий ID не може бути меншим за початковий.")
        self._current = start
        self._stop = stop


    def __iter__(self) -> CarIdIterator:
        return self


    def __next__(self) -> int:
        if self._stop is not None and self._current >= self._stop:
            raise StopIteration
        current = self._current
        self._current += 1
        return current
```

Поведінку підтверджує перший фрагмент виводу в розділі 22: перший прохід повертає `[1, 2, 3]`, другий — порожній список `[]`.

10. Приклад generator function та yield

Функція, що містить `yield`, є generator function: її виклик не виконує тіло, а створює generator object. Тіло виконується порціями — під час `next()` чи споживання у циклі; `yield` повертає значення, призупиняє функцію й зберігає локальні змінні та позицію виконання. `return` же завершує функцію остаточно. У проєкті generator functions — це `clean_lines()`, `parse_csv_rows()`, `validate_car_rows()`, `filter_cars()`,

`format_car_rows()`, `batched()`, `flatten_batches()`, `group_cars_by_brand()`, `infinite_car_ids()` та інші.

```
def clean_lines(lines: Iterable[str]) -> Iterator[str]:
    """Обрізає пробіли й пропускає порожні рядки, не матеріалізуючи потік."""
    for line in lines:
        cleaned = line.strip()
        if cleaned:
            yield cleaned
```

```
def filter_cars(
    cars: Iterable[CarRecord],
    brand: str | None = None,
    year: int | None = None,
) -> Iterator[CarRecord]:
    """Фільтрує за маркою без урахування регістру та/або точним роком."""
    normalized_brand = brand.strip().casefold() if brand is not None else None
    for car in cars:
        if normalized_brand is not None and car.brand.casefold() != normalized_brand:
            continue
        if year is not None and car.year != year:
            continue
        yield car
```

`filter_cars()` не створює список результатів: кожен автомобіль, що пройшов перевірку, передається споживачеві через `yield`, а решта файлу не читається, доки не запитано наступний елемент. Нескінченний генератор `infinite_car_ids()` показує, що `yield` дозволяє описувати й нескінченні послідовності — їх потрібно обмежувати (розділ 13):

```
def infinite_car_ids(start: int = 1) -> Iterator[int]:
    """Генерує нескінченний потік ID; споживач має обмежити його islice."""
    if start < 1:
        raise ValueError("Початковий ID має бути додатним.")
    yield from count(start)


def first_car_ids(limit: int, start: int = 1) -> Iterator[int]:
    """Безпечно бере скінченний префікс нескінченного потоку."""
    if limit < 0:
        raise ValueError("Ліміт має бути невід'ємним.")
    return islice(infinite_car_ids(start), limit)
```

11. Приклад yield from

`yield from` делегує видачу значень іншому iterable або generator, замінюючи внутрішній цикл `for item in group: yield item`. У проєкті цю конструкцію використано в чотирьох місцях: `read_lines()` передає рядки файлу, `parse_csv_rows()` — словники `DictReader`, `flatten_batches()` — елементи пакетів, `flatten_nested()` — рекурсивно розгортає вкладені списки.

```
def read_lines(path: Path) -> Iterator[str]:
    """Читає файл построково та закриває його після вичерпання генератора."""
    with path.open("r", encoding="utf-8", newline="") as source:
        yield from source
```

```
def parse_csv_rows(lines: Iterable[str]) -> Iterator[CsvRow]:
    """Перетворює CSV-рядки на словники за назвами колонок."""
    yield from csv.DictReader(lines, skipinitialspace=True)
```

```
def flatten_nested(values: object) -> Iterator[object]:
    """Рекурсивно розгортає вкладені списки й кортежі через yield from."""
    if isinstance(values, (list, tuple)):
        for value in values:
            yield from flatten_nested(value)
    else:
        yield values


def flatten_batches(batches: Iterable[Iterable[T]]) -> Iterator[T]:
    """Об'єднує пакети через yield from без створення спільного списку."""
    for batch in batches:
        yield from batch
```

Окремо варто відзначити, що `with` усередині `read_lines()` лишається активним, доки generator споживається: файл закривається, лише коли generator вичерпано або закрито (`close()`), тому споживач має довести потік до кінця або явно його закрити.

12. Generator expression

Generator expression має вигляд `(вираз for елемент in iterable if умова)` і, на відміну від list comprehension, не створює колекцію, а обчислює елементи на запит. У проєкті вони застосовані в `summarize_cars()` (перетворення запису) та `running_total_prices()` / `consecutive_price_changes()` (потік цін без проміжного списку):

```
def summarize_cars(cars: Iterable[CarRecord]) -> Iterator[CarSummary]:
    """Створює короткі звіти про автомобілі, коли їх запитує споживач."""
    return (
        CarSummary(car.id, car.full_name, car.year, car.price)
        for car in cars
    )
```

```
# analytics.py
def running_total_prices(cars):
    return accumulate(car.price for car in cars)


def consecutive_price_changes(cars):
    prices = (car.price for car in cars)
    return pairwise(prices)
```

Різниця з eager-варіантом: `[x * 2 for x in range(1_000_000)]` одразу матеріалізує мільйон елементів, а `(x * 2 for x in range(1_000_000))` створює лише невеликий об’єкт-генератор. Побічний ефект lazy-підходу — помилки в такому виразі виникають під час споживання, а не під час його створення.

13. Використані засоби itertools

Модуль `itertools` надає оптимізовані lazy-«будівельні блоки» для ітерації. У лабораторній роботі використано шість з них:

Засіб	Призначення	Де застосовано
`chain.from_iterable`	послідовно об'єднує кілька iterable	`iter_car_records()` — обробка кількох CSV-файлів одним потоком
`islice`	бере частину iterator без матеріалізації	`first_matching_cars()`, `batched()`, `first_car_ids()`
`count`	нескінченна арифметична послідовність	`infinite_car_ids()`
`accumulate`	накопичувальні значення	`running_total_prices()` — наростаюча вартість
`pairwise`	пари сусідніх елементів	`consecutive_price_changes()` — зміна цін
`groupby`	групує послідовні елементи з однаковим ключем	`group_cars_by_brand()` після `sorted`

```
# pipeline.py — кілька файлів як один потік
rows = chain.from_iterable(
    parse_csv_rows(clean_lines(read_lines(path)))
    for path in paths
)


# filters.py — перші N збігів, читання припиняється достроково
return islice(filter_cars(cars, brand=brand, year=year), limit)


# analytics.py — groupby вимагає попереднього сортування за тим самим ключем
ordered_cars = sorted(cars, key=lambda car: car.brand.casefold())
for brand, group in groupby(ordered_cars, key=lambda car: car.brand):
    yield brand, list(group)
```

Важливо, що `groupby` групує лише **сусідні** елементи, тому без сортування одна марка могла б з'явитися в кількох групах. Сортування вимагає O(n log n) часу та O(n) пам'яті, тож `group_cars_by_brand()` — свідомий виняток із потокової моделі, про що зазначено в аналізі складності.

14. Схема pipeline

Етапи конвеєра з'єднано послідовно; кожен етап є generator і запитує наступний елемент у попереднього лише тоді, коли споживач просить результат:

```
data/cars.csv (або кілька CSV через chain)
      │
      ▼   read_lines()          — построкове читання файлу  (yield from)
```

```
      ▼   clean_lines()        — strip, пропуск порожніх рядків
      ▼   parse_csv_rows()     — csv.DictReader → dict
      ▼   validate_car_rows()  — типізація, перевірка, облік помилок → CarRecord
      ▼   filter_cars()        — марка / рік
      ▼   summarize_cars() / format_car_rows()   — transformation
      ▼   batched() / calculate_statistics() / find_car_by_id()   — sink
      │
      ▼
   результат (статистика, перші N записів, пакети)
```

Допоки споживач не викличе `next()`, `sum()`, `list()` чи цикл `for`, файл фактично не читається: `build_pipeline()` повертає лише generator object. Pipeline є одноразовим — для повторного проходу його потрібно створити заново.

15. Код streaming reader

15.1. Файл streaming/readers.py

```
"""Потокове читання вхідних файлів."""


from collections.abc import Iterable, Iterator
from pathlib import Path



def read_lines(path: Path) -> Iterator[str]:
    """Читає файл построково та закриває його після вичерпання генератора."""
    with path.open("r", encoding="utf-8", newline="") as source:
        yield from source



def clean_lines(lines: Iterable[str]) -> Iterator[str]:
    """Обрізає пробіли й пропускає порожні рядки, не матеріалізуючи потік."""
    for line in lines:
        cleaned = line.strip()
        if cleaned:
            yield cleaned
```

15.2. Файл streaming/parsers.py

```
"""Потоковий розбір CSV-записів."""


import csv
from collections.abc import Iterable, Iterator


CsvRow = dict[str | None, object]



def parse csv rows(lines: Iterable[str]) -> Iterator[CsvRow]:
    """Перетворює CSV-рядки на словники за назвами колонок."""
    yield from csv.DictReader(lines, skipinitialspace=True)
```

`read_lines()` відкриває файл у режимі читання та передає рядки через `yield from`; весь файл ніколи не завантажується (на відміну від `file.read()`, що поміщає все в RAM). `newline=""` залишає обробку розривів рядків модулю

`csv. clean_lines()` — окремий етап очищення, а `parse_csv_rows()` перетворює рядки на словники за назвами колонок.

16. Validation stage

Етап валідації перетворює «сирі» словники на типізовані `CarRecord`, а некоректні рядки не зупиняють потік: вони пропускаються й обліковуються в `ValidationStats` із категорією причини (`invalid year`, `missing mileage`, `unexpected extra fields` тощо).

```
"""Перевірка та типізація рядків каталогу."""


import math
from collections.abc import Iterable, Iterator


from car_catalog.streaming.models import CarRecord, ValidationStats
from car_catalog.streaming.parsers import CsvRow


_REQUIRED_FIELDS = ("id", "brand", "model", "year", "price", "mileage")



class  InvalidCarRow(ValueError):
    def __init__(self, reason: str) -> None:
        super().__init__(reason)
        self.reason = reason



def validate_car_rows(
    rows: Iterable[CsvRow],
    stats: ValidationStats | None = None,
) -> Iterator[CarRecord]:
    """Видає валідні записи, підраховуючи кожен відхилений рядок і причину."""
    for row in rows:
        try:
            if None in row:
                raise _InvalidCarRow("unexpected extra fields")
            values: dict[str, str] = {}
            for field in _REQUIRED_FIELDS:
                value = row.get(field)
                if not isinstance(value, str):
                    raise _InvalidCarRow(f"missing {field}")
                values[field] = value.strip()


            try:
                car_id = int(values["id"])
            except ValueError as error:
                raise _InvalidCarRow("invalid id") from error
            brand = values["brand"]
            model = values["model"]
            try:
                year = int(values["year"])
            except ValueError as error:
                raise  InvalidCarRow("invalid year") from error
            try:
                price = float(values["price"])
            except ValueError as error:
                raise _InvalidCarRow("invalid price") from error
            try:
                mileage = int(values["mileage"])
            except ValueError as error:
                raise _InvalidCarRow("invalid mileage") from error


            if car_id < 1:
                raise _InvalidCarRow("id must be positive")
```

```
            if not brand or not model:
                raise _InvalidCarRow("brand and model must be non-empty")
            if year < 1886:
                raise _InvalidCarRow("year must be at least 1886")
            if not math.isfinite(price) or price < 0:
                raise _InvalidCarRow("price must be finite and non-negative")
            if mileage < 0:
                raise _InvalidCarRow("mileage must be non-negative")
        except (AttributeError, TypeError, ValueError, OverflowError) as error:
            if stats is not None:
                stats.invalid_count += 1
                reason = error.reason if isinstance(error, _InvalidCarRow) else
type(error).__name__
                stats.invalid_reasons[reason] += 1
            continue


        if stats is not None:
            stats.valid count += 1
        yield CarRecord(car_id, brand, model, year, price, mileage)
```

```
@dataclass(slots=True)
class ValidationStats:
    """Кількість коректних і відхилених CSV-записів."""


    valid_count: int = 0
    invalid_count: int = 0
    invalid_reasons: Counter[str] = field(default_factory=Counter)
```

17. Filtering stage

```
"""Lazy-фільтрація записів каталогу."""


from collections.abc import Iterable, Iterator
from itertools import islice


from car_catalog.streaming.models import CarRecord



def filter_cars(
    cars: Iterable[CarRecord],
    brand: str | None = None,
    year: int | None = None,
) -> Iterator[CarRecord]:
    """Фільтрує за маркою без урахування регістру та/або точним роком."""
    normalized_brand = brand.strip().casefold() if brand is not None else None
    for car in cars:
        if normalized_brand is not None and car.brand.casefold() != normalized_brand:
            continue
        if year is not None and car.year != year:
            continue
        yield car



def first_matching_cars(
    cars: Iterable[CarRecord],
    limit: int,
    brand: str | None = None,
    year: int | None = None,
) -> Iterator[CarRecord]:
    """Повертає не більше перших N збігів і припиняє читання достроково."""
    if limit < 0:
        raise ValueError("Ліміт має бути невід'ємним.")
    return islice(filter_cars(cars, brand=brand, year=year), limit)
```

```
def find_car_by_id(cars: Iterable[CarRecord], car_id: int) -> CarRecord | None:
    """Знаходить запис за ID, завершуючи читання відразу після збігу."""
    return next((car for car in cars if car.id == car_id), None)
```

`filter_cars()` порівнює марку без урахування регістру (`casefold()`) і/або рік. `first_matching_cars()` додає `islice`, тому читання файлу припиняється одразу після N-го збігу. `find_car_by_id()` використовує `next()` з generator expression і значенням за замовчуванням `None`: пошук завершується на першому збігу, що є прикладом early termination (розділ 26).

18. Transformation stage

```
"""Lazy-перетворення потокових записів."""


import csv
from collections.abc import Iterable, Iterator
from io import StringIO


from car_catalog.streaming.models import CarRecord, CarSummary



def summarize_cars(cars: Iterable[CarRecord]) -> Iterator[CarSummary]:
    """Створює короткі звіти про автомобілі, коли їх запитує споживач."""
    return (
        CarSummary(car.id, car.full_name, car.year, car.price)
        for car in cars
    )



def format_car_rows(cars: Iterable[CarRecord]) -> Iterator[str]:
    """Формує CSV-подібні рядки експорту без проміжного списку."""
    for car in cars:
        buffer = StringIO()
        csv.writer(buffer, lineterminator="").writerow(
            (car.id, car.brand, car.model, car.year, f"{car.price:.2f}", car.mileage)
        )
        yield buffer.getvalue()
```

`summarize_cars()` перетворює повний запис на короткий `CarSummary` (id, назва, рік, ціна), а `format_car_rows()` формує рядки CSV-експорту, використовуючи `csv.writer` у буфері `StringIO`: потокове перетворення без проміжного списку.

19. Aggregation stage

Агрегація виконується за **один прохід** потоку з O(1) додаткової пам'яті (не рахуючи словник частот марок). `calculate_statistics()` одночасно накопичує кількість, суму цін, найдорожчий автомобіль, автомобіль з найменшим пробігом і лічильник марок (`Counter`).

```
"""Потокова агрегація та операції itertools над записами автомобілів."""
```

```
from collections import Counter
from collections.abc import Iterable, Iterator
from itertools import accumulate, groupby, pairwise


from car_catalog.streaming.models import CarRecord, CarStatistics



def streaming_average(values: Iterable[float]) -> float:
    """Обчислює середнє за один прохід із O(1) пам'яттю."""
    total = 0.0
    count = 0
    for value in values:
        total += value
        count += 1
    return total / count if count else 0.0



def calculate statistics(cars: Iterable[CarRecord]) -> CarStatistics:
    """Обчислює лічильник, середню ціну, min/max і марки за один прохід."""
    count = 0
    total price = 0.0
    most_expensive: CarRecord | None = None
    lowest_mileage: CarRecord | None = None
    brand counts: Counter[str] = Counter()


    for count, car in enumerate(cars, start=1):
        total_price += car.price
        brand_counts[car.brand] += 1
        if most expensive is None or car.price > most expensive.price:
            most_expensive = car
        if lowest_mileage is None or car.mileage < lowest_mileage.mileage:
            lowest mileage = car


    return CarStatistics(
        count=count,
        average_price=total_price / count if count else 0.0,
        most_expensive=most_expensive,
        lowest_mileage=lowest_mileage,
        brand_counts=dict(brand_counts),
    )



def running_total_prices(cars: Iterable[CarRecord]) -> Iterator[float]:
    """Видає накопичувальну суму цін через itertools.accumulate."""
    return accumulate(car.price for car in cars)



def consecutive_price_changes(
    cars: Iterable[CarRecord],
) -> Iterator[tuple[float, float]]:
    """Видає пари послідовних цін для аналізу змін через itertools.pairwise."""
    prices = (car.price for car in cars)
    return pairwise(prices)



def group_cars_by_brand(
    cars: Iterable[CarRecord],
) -> Iterator[tuple[str, list[CarRecord]]]:
    """Сортує записи за маркою та групує сусідні записи через itertools.groupby."""
    ordered cars = sorted(cars, key=lambda car: car.brand.casefold())
    for brand, group in groupby(ordered cars, key=lambda car: car.brand):
        yield brand, list(group)
```

```
@dataclass(frozen=True, slots=True)
class CarStatistics:
    """Агрегована статистика, побудована одним проходом потоку."""
```

```
    count: int
    average_price: float
    most_expensive: CarRecord | None
    lowest_mileage: CarRecord | None
    brand_counts: dict[str, int]
```

20. Batching

`batched()` ділить будь-який iterable на послідовні списки заданого розміру за допомогою `islice`; в пам'яті одночасно перебуває лише поточний пакет (O(b)). Умова `while batch := list(islice(...))` завершує цикл, коли iterator вичерпано; останній пакет може бути коротшим.

```
"""Обробка потоків пакетами."""


from collections.abc import Iterable, Iterator
from itertools import islice
from typing import TypeVar


T = TypeVar("T")



def batched(items: Iterable[T], batch_size: int) -> Iterator[list[T]]:
    """Ділить довільний iterable на послідовні пакети заданого розміру."""
    if batch_size < 1:
        raise ValueError("Розмір пакета має бути не меншим за 1.")


    iterator = iter(items)
    while batch := list(islice(iterator, batch_size)):
        yield batch



def flatten_batches(batches: Iterable[Iterable[T]]) -> Iterator[T]:
    """Об'єднує пакети через yield from без створення спільного списку."""
    for batch in batches:
        yield from batch
```

Пакетна обробка корисна для зовнішніх API, баз даних і файлових операцій, де ефективніше передавати дані блоками. Наприклад, для шести записів `cars.csv` і розміру пакета 4 `batched()` повертає `[[1, 2, 3, 4], [5, 6]]` (розділ 22).

21. Збирання pipeline

```
"""Побудова lazy pipeline для одного або кількох CSV-файлів."""


from collections.abc import Iterable, Iterator
from itertools import chain
from pathlib import Path


from car_catalog.streaming.filters import filter_cars
from car_catalog.streaming.models import CarRecord, ValidationStats
from car_catalog.streaming.parsers import parse_csv_rows
from car_catalog.streaming.readers import clean_lines, read_lines
```

```
from car_catalog.streaming.validation import validate_car_rows



def iter_car_records(
    paths: Iterable[Path],
    stats: ValidationStats | None = None,
) -> Iterator[CarRecord]:
    """Потоково читає, очищує, розбирає та валідує CSV з одного чи кількох файлів."""
    rows = chain.from_iterable(
        parse_csv_rows(clean_lines(read_lines(path)))
        for path in paths
    )
    yield from validate_car_rows(rows, stats)



def build_pipeline(
    paths: Iterable[Path],
    brand: str | None = None,
    year: int | None = None,
    stats: ValidationStats | None = None,
) -> Iterator[CarRecord]:
    """Збирає lazy pipeline: source → parse → validate → filter."""
    return filter_cars(iter_car_records(paths, stats), brand=brand, year=year)
```

`iter_car_records()` з'єднує етапи читання, очищення, розбору й валідації для одного чи кількох файлів, а `build_pipeline()` додає фільтрацію. Усе складено з generator functions, тому повернутий об'єкт не виконує жодної роботи до моменту споживання. Демонстраційний запуск `demo.py` показує агрегацію, фільтрацію, перетворення, batching і групування:

```
def run_demo() -> None:
    """Показує агрегацію, фільтрацію, трансформацію та batching."""
    path = sample_data_path()
    validation = ValidationStats()
    statistics = calculate_statistics(build_pipeline([path], stats=validation))
    print(
        f"Оброблено {statistics.count} автомобілів; "
        f"відхилено рядків: {validation.invalid_count}"
    )
    print(f"Середня ціна: ${statistics.average_price:,.2f}")
    if statistics.most_expensive is not None:
        print(f"Найдорожчий: {statistics.most_expensive.full_name}")
    if statistics.lowest_mileage is not None:
        print(f"Найменший пробіг: {statistics.lowest_mileage.full_name}")


    recent = build_pipeline([path], brand="Toyota", year=2020)
    print("Перші збіги:", list(islice(summarize_cars(recent), 3)))


    records = build_pipeline([path])
    print("Перший пакет:", next(batched(records, 3), []))
    groups = group_cars_by_brand(build_pipeline([path]))
    print("Групи марок:", [(brand, len(cars)) for brand, cars in groups])
```

22. Результат роботи програми

Нижче наведено вивід реальних запусків: демонстрація конвеєра (`python -m car_catalog.streaming` або пункт 11 меню), приклади роботи iterator protocol і `itertools` та результат тестів.

Рисунок 1 — Демонстрація потокового конвеєра на data/cars.csv

```
Оброблено 6 автомобілів; відхилено рядків: 1
Середня ціна: $53,166.67
Найдорожчий: Porsche 911 Carrera
Найменший пробіг: Porsche 911 Carrera
Перші збіги: [CarSummary(id=2, name='Toyota Camry', year=2020, price=25000.0)]
Перший пакет: [CarRecord(id=1, brand='Tesla', model='Model Y', year=2023, price=50000.0,
mileage=12000), CarRecord(id=2, brand='Toyota', model='Camry', year=2020, price=25000.0,
mileage=45000), CarRecord(id=3, brand='Honda', model='Civic', year=2021, price=22000.0,
mileage=30000)]
Групи марок: [('BMW', 1), ('Honda', 1), ('Porsche', 1), ('Tesla', 1), ('Toyota', 2)]
```

Рисунок 1

Рядок `invalid,Unknown,Model,2020,not-a-price,100` відхилено на етапі валідації й не вплинув на статистику: оброблено 6 автомобілів, середня ціна $53 166,67.

Рисунок 2 — Iterator protocol, yield from, batching та itertools

```
from pathlib import Path


from car_catalog.streaming.analytics import (
    consecutive_price_changes,
    running_total_prices,
    streaming_average,
)
from car_catalog.streaming.batches import batched, flatten_batches
from car_catalog.streaming.filters import find_car_by_id
from car_catalog.streaming.iterators import CarIdIterator, first_car_ids, flatten_nested
from car_catalog.streaming.pipeline import build_pipeline


ids = CarIdIterator(1, 4)
print("iter(ids) is ids:", iter(ids) is ids)
print("перший прохід:", list(ids))
print("другий прохід:", list(ids))
print("нескінченний потік + islice:", list(first_car_ids(5, start=100)))
print("yield from (flatten):", list(flatten_nested([1, [2, 3], [4, [5, 6]]])))


path = Path("data/cars.csv")
print("batched:", [[car.id for car in batch] for batch in batched(build_pipeline([path]), 4)])
print("flatten_batches:", list(flatten_batches([[1, 2], [3], [4, 5]])))
print("accumulate:", list(running_total_prices(build_pipeline([path]))))
print("pairwise:", list(consecutive_price_changes(build_pipeline([path])))[:2])
print("streaming_average:", streaming_average(car.price for car in build_pipeline([path])))
print("find_car_by_id(3):", find_car_by_id(build_pipeline([path]), 3))
pipeline = build_pipeline([path])
print("pipeline — generator object:", type(pipeline).__name__)
```

```
iter(ids) is ids: True
перший прохід: [1, 2, 3]
другий прохід: []
нескінченний потік + islice: [100, 101, 102, 103, 104]
yield from (flatten): [1, 2, 3, 4, 5, 6]
batched: [[1, 2, 3, 4], [5, 6]]
flatten_batches: [1, 2, 3, 4, 5]
accumulate: [50000.0, 75000.0, 97000.0, 115000.0, 187000.0, 319000.0]
pairwise: [(50000.0, 25000.0), (25000.0, 22000.0)]
streaming_average: 53166.666666666664
find car by id(3): CarRecord(id=3, brand='Honda', model='Civic', year=2021, price=22000.0,
mileage=30000)
pipeline — generator object: generator
```

Рисунок 2

Рисунок 3 — Результат запуску pytest

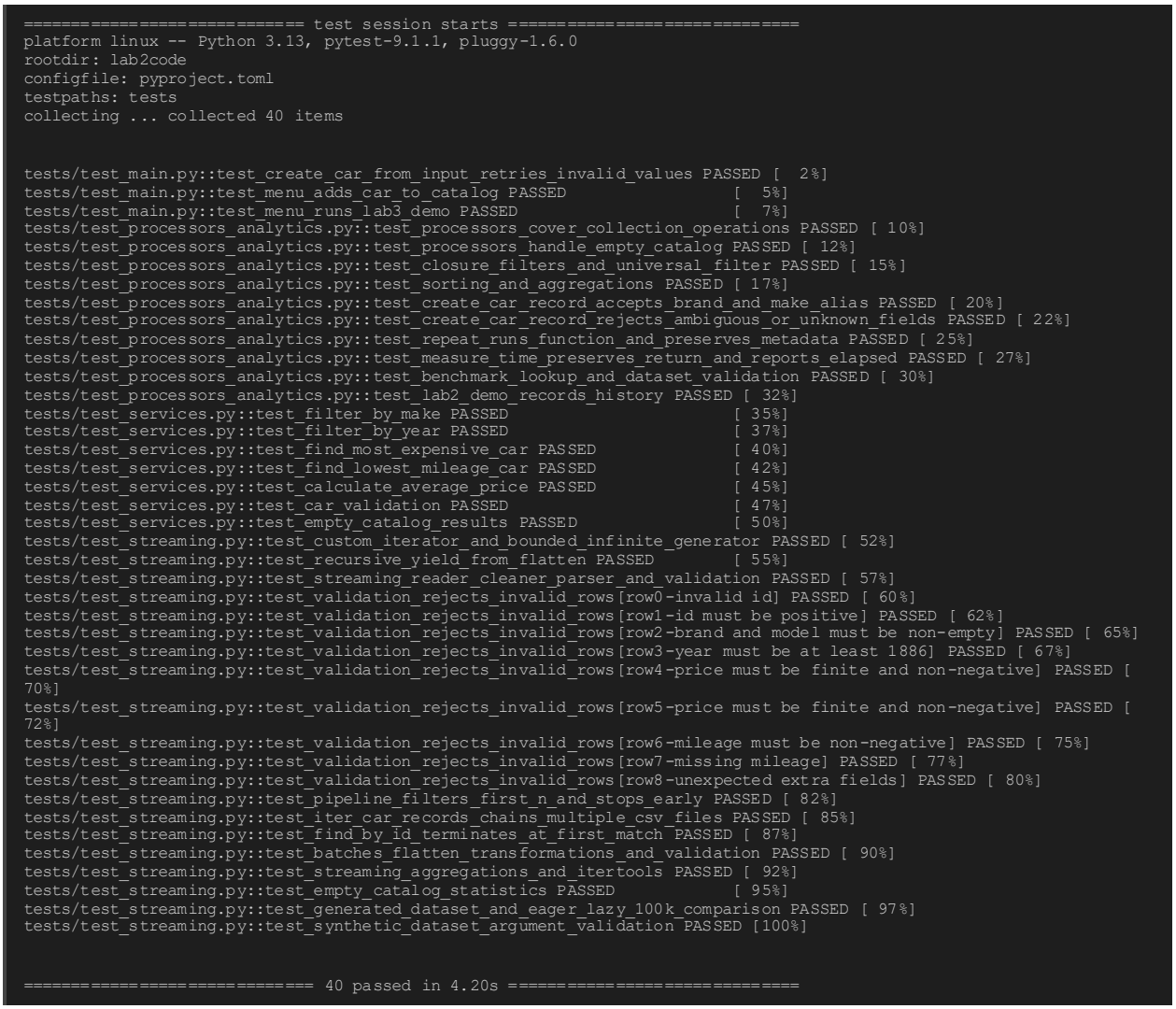


Рисунок 3

Усі 40 тестів успішні: 20 тестів ЛР1–ЛР2 і 20 нових тестів ЛР3 (`tests/test_streaming.py`). Вони перевіряють власний iterator та обмежений нескінченний generator, рекурсивний `yield from`, читання, очищення, розбір і валідацію (дев'ять варіантів некоректних рядків), фільтрацію з раннім завершенням, об'єднання кількох файлів, пошук за ID, batching, агрегацію та `itertools`, порожній вхід і порівняння eager/lazy на 100 000 записів.

23. Eager implementation

Eager-варіант спочатку читає весь файл як один рядок (`read_text()`), розбирає його та **матеріалізує всі валідні записи в список**, і лише потім

обчислює статистику. Усі дані одночасно перебувають у пам'яті; їх зручно використовувати повторно, але обсяг пам'яті зростає разом із розміром файлу.

```
def _process_eager(path: Path) -> tuple[CarStatistics, ValidationStats]:
    contents = path.read_text(encoding="utf-8")
    rows = parse_csv_rows(StringIO(contents))
    validation = ValidationStats()
    cars = list(validate_car_rows(rows, validation))
    return calculate_statistics(cars), validation
```

24. Lazy implementation

Lazy-варіант не зберігає ні тексту файлу, ні списку записів: `calculate_statistics()` споживає потік `iter_car_records()` запис за записом, тому в пам'яті одночасно перебуває лише поточний рядок і невеликі лічильники.

```
def  process lazy(path: Path) -> tuple[CarStatistics, ValidationStats]:
    validation = ValidationStats()
    statistics = calculate_statistics(iter_car_records([path], validation))
    return statistics, validation
```

Функція `compare_processing()` запускає обидва режими на одному файлі й перевіряє, що агрегати (кількість, середня ціна, найдорожчий автомобіль, мінімальний пробіг, лічильники марок) та кількості валідних і некоректних рядків збігаються, а вже потім повертає вимірювання часу, пікової пам'яті (`tracemalloc`) і часу до першого запису.

25. Результати експерименту

Експеримент виконано для наборів 10 000, 100 000 та 500 000 валідних автомобілів (плюс по одному некоректному рядку на кожні 10 000). Для кожного набору порівняно eager і lazy обробку за повним часом, піковою пам'яттю та часом до першого запису. Експеримент запускається скриптом, що використовує функції з `benchmark.py`:

```
"""Експеримент ЛР3: eager vs lazy для 10 000, 100 000 і 500 000 записів + early termination."""
import tempfile
from pathlib import Path
from time import perf_counter


from car_catalog.streaming.benchmark import compare_processing, write_synthetic_dataset
from car_catalog.streaming.filters import find_car_by_id
from car_catalog.streaming.pipeline import iter_car_records


rows = []
with tempfile.TemporaryDirectory() as d:
    for size in (10_000, 100_000, 500_000):
        p = Path(d) / f"cars_{size}.csv"
        write_synthetic_dataset(p, size, invalid_every=10_000)
        e, l = compare_processing(p)
        rows.append(dict(size=size, e_t=e.elapsed_seconds, l_t=l.elapsed_seconds,
                         e_m=e.peak_memory_bytes/2**20, l_m=l.peak_memory_bytes/2**20,
                         e_f=e.time_to_first_record_seconds, l_f=l.time_to_first_record_seconds,
```

```
                        valid=l.validation.valid_count, invalid=l.validation.invalid_count,
                        mb=p.stat().st_size/2**20))
        print(rows[-1], flush=True)
    # early termination on 1 000 000 records: find the car with id 10
    p = Path(d) / "cars_1m.csv"
    write_synthetic_dataset(p, 1_000_000)
    t = perf_counter(); lazy_car = find_car_by_id(iter_car_records([p]), 10); lazy_t = perf_counter()
- t
    t = perf_counter(); eager_all = list(iter_car_records([p])); eager_car = next(c for c in
eager_all if c.id == 10); eager_t = perf_counter() - t
    assert lazy_car == eager_car
    et = dict(lazy=lazy_t, eager=eager_t, car=str(lazy_car))
    print(et)
```

Скрипт можна зберегти, наприклад, як `experiments/lab3_experiment.py` і запускати командою `python experiments/lab3_experiment.py`. Для швидкої перевірки на 100 000 записів достатньо `python -m car_catalog.streaming.benchmark` (пункт 12 меню):

Рисунок 4 — Вивід benchmark на 100 000 записів

```
Набір: 100,000 валідних авто плюс некоректні тестові рядки
Режим    |       Час, с |   Peak memory, MiB |   До 1-го запису, с |    Валідні |  Некоректні
-------------------------------------------------------------------------------------------------
eager    |       1.7041 |            52.410 |           0.025597 |    100,000 |          10
lazy     |       1.7150 |             0.055 |           0.000234 |    100,000 |          10


Висновок: lazy pipeline не зберігає весь CSV та підтримує раннє завершення; eager обробка може
бути простішою й швидшою на малих даних.
```

Рисунок 4

Таблиця 1 — Elapsed time

Кількість records	Розмір файлу, MiB	Eager time, с	Lazy time, с
10 000	0.435	0.1720	0.1558
100 000	4.607	1.8401	1.8184
500 000	23.897	9.3043	8.6510

Таблиця 2 — Peak memory (tracemalloc)

Кількість records	Eager peak memory, MiB	Lazy peak memory, MiB	Різниця, разів
10 000	5.122	0.045	≈ 114
100 000	52.405	0.055	≈ 958
500 000	266.883	0.055	≈ 4 876

Таблиця 3 — Time to first result

Кількість records	Eager, с	Lazy, с	Різниця, разів
10 000	0.002041	0.000160	≈ 13
100 000	0.021664	0.000223	≈ 97
500 000	0.092731	0.000254	≈ 364

Усі значення — з одного запуску; точні цифри залежать від комп'ютера, операційної системи, дискового кешу та навантаження, тому характер залежностей важливіший за абсолютні значення. `tracemalloc` вимірює лише Python-аллокації (не всю пам'ять процесу) й сам уповільнює виконання, тож повний час у таблиці 1 включає цей додатковий overhead в обох режимах.

26. Early termination, нескінченні потоки та типові помилки

26.1. Early termination

Для набору з 1 000 000 записів знайдено автомобіль з `id = 10`. Lazy-варіант (`find_car_by_id(iter_car_records([path]), 10)`) прочитав лише початок файлу й завершився за **0.000212 с**. Eager-варіант спочатку сформував повний список із мільйона записів (`list(iter_car_records([path]))`) і лише потім виконав пошук — **3.829 с**, тобто приблизно в 18 032 разів повільніше. Знайдений запис в обох випадках однаковий: `CarRecord(id=10, brand='Brand_10', model='Model_10', year=2010, price=10010.0, mileage=70)`.

26.2. Нескінченний потік

`infinite_car_ids()` генерує ID без кінця, тому `list(infinite_car_ids())` ніколи не завершиться й поступово вичерпає пам'ять. Безпечний спосіб — обмежити потік `islice`: `first_car_ids(5, start=100)` повертає `[100, 101, 102, 103, 104]` (рисунок 2).

26.3. Типові помилки

Помилка	Наслідок	Як уникнуто в проєкті
повторне використання вичерпаного generator	другий прохід повертає порожній результат	pipeline створюється заново для кожного споживача (`demo.py`)
випадкова матеріалізація `list(generator)`	втрачається перевага пам'яті	агрегація й фільтрація працюють з потоком; список лише для малих результатів
нескінченний iterator без обмеження	програма не завершується	`first_car_ids()` використовує `islice`

Помилка	Наслідок	Як уникнуто в проєкті
lazy exception	помилка виникає під час споживання, а не створення	валідація не піднімає виняток назовні, а облікує й пропускає рядок
невикористаний ресурс generator	файл лишається відкритим	`with` у `read_lines()`; `records.close()` у `_lazy_time_to_first()`

27. Аналіз результатів

Пам'ять. Пікова пам'ять eager-варіанта зростає лінійно з розміром набору: 5.122 → 52.405 → 266.883 MiB для 10 000, 100 000 і 500 000 записів, що значно більше за розмір самого файлу (23.897 MiB для 500 000 записів), бо кожен запис — це кілька Python-об'єктів. Lazy-варіант стабільно використовує приблизно 0.055 MiB незалежно від розміру набору (O(1)), тож на 500 000 записів різниця сягає ≈ 4 876 разів.

Час до першого результату. Eager-варіант мусить прочитати й обробити весь файл, перш ніж видати перший запис, тому цей час зростає з розміром набору (0.002041 → 0.092731 с); у lazy-варіанті він практично сталий (близько 0.000254 с).

Повний час. Повний час обох режимів зіставний (8.6510 с проти 9.3043 с на 500 000 записів): різниця в межах кількох відсотків і не дає підстав стверджувати, що generator «завжди швидший». Загальна робота — читання, розбір CSV, валідація — однакова в обох варіантів; lazy виграє насамперед пам'яттю, раннім результатом і можливістю раннього завершення.

27.1. Переваги lazy evaluation

- мінімальний peak memory: розмір набору практично не впливає на споживання пам'яті;
- швидкий доступ до перших результатів і можливість early termination (на 1 000 000 записів — у тисячі разів швидше);
- робота з нескінченними та дуже великими потоками, які не вміщуються в RAM;
- композиційність: етапи конвеєра незалежні, їх легко тестувати й комбінувати.

27.2. Недоліки lazy evaluation

- вичерпаний generator не можна обійти повторно — потрібно створити pipeline наново;
- відсутні `len()`, індексація та випадковий доступ без матеріалізації;
- помилки виникають під час споживання, що ускладнює налагодження;
- відкриті ресурси (файл) живуть, поки generator не вичерпано або не закрито;
- для малих наборів та повторних обходів eager-варіант простіший і не поступається за часом;
- операції, що потребують усіх даних (сортування для `groupby`), все одно матеріалізують потік.

28. Аналіз складності

Операція	Час	Додаткова пам'ять
читання, очищення, parsing, validation	O(n)	O(1) на запис
фільтрація за маркою / роком	O(n)	O(1)
пошук за ID, перші N збігів	до O(n), за раннього збігу менше	O(1) / O(N) для результату
streaming average, min, max, лічильники	O(n)	O(1) + O(u) для u марок
batching (пакет розміру b)	O(n) загалом	O(b)
`accumulate`, `pairwise`	O(n)	O(1)
сортування перед `groupby`	O(n log n)	O(n)
eager читання та агрегація	O(n)	O(n)

29. Завдання підвищеної складності

Для максимальної оцінки реалізовано більше трьох додаткових можливостей із розділу 70 завдання:

Можливість	Реалізація
власний iterator class	`CarIdIterator`
recursive generator із `yield from`	`flatten_nested()`
infinite generator + `islice`	`infinite_car_ids()`, `first_car_ids()`
`itertools.chain`	`iter_car_records()` — кілька CSV-файлів

Можливість	Реалізація
`itertools.accumulate`	`running_total_prices()`
`itertools.pairwise`	`consecutive_price_changes()`
`itertools.groupby`	`group_cars_by_brand()`
pipeline із декількох files	`build_pipeline(paths)`
streaming export	`format_car_rows()`
підрахунок invalid records	`ValidationStats` із причинами відхилення
measurement time-to-first-result	`ProcessingMeasurement.time_to_first_record_seconds`
lazy search із early termination	`find_car_by_id()`, `first_matching_cars()`
comparison of memory for list vs generator	`compare_processing()`, таблиця 2

30. Відповіді на контрольні запитання

1. Що таке iterable?

Об'єкт, з якого можна отримати iterator функцією `iter()`: список, кортеж, словник, файл.

2. Що таке iterator?

Об'єкт, що зберігає стан обходу та за викликом `next()` повертає наступний елемент, а по завершенні піднімає `StopIteration`.

3. Чим iterable відрізняється від iterator?

Iterable створює iterator і може обходитися багаторазово; iterator сам видає елементи, зберігає позицію й зазвичай одноразовий.

4. Для чого використовується iter()?

Повертає iterator для переданого iterable (викликає його `__iter__`).

5. Для чого використовується next()?

Запитує в iterator наступний елемент; може приймати значення за замовчуванням, як у `find_car_by_id()`.

6. Що робить __iter__()?

Повертає iterator; в iterator-класах повертає `self` (`CarIdIterator.__iter__`).

7. Що робить __next__()?

Повертає наступний елемент або піднімає `StopIteration`, коли елементи закінчилися.

8. Що таке StopIteration?

Виняток-сигнал про вичерпання iterator; `for` перехоплює його й завершує цикл.

9. Як цикл for використовує iterator protocol?

Викликає `iter()` один раз, потім багаторазово `next()` і зупиняється на `StopIteration`.

10. Чому iterator часто називають one-shot object?

Він зберігає позицію й не повертається на початок: другий `list(iterator)` повертає `[]`.

11. Чому iter(iterator) is iterator?

Iterator реалізує протокол сам, тому його `__iter__()` повертає `self`.

12. Чи можна повторно використати exhausted generator?

Ні, потрібно створити новий generator (у проєкті pipeline збирається заново для кожного споживача).

13. Що таке generator function?

Функція з `yield`: її виклик повертає generator object, а тіло виконується під час споживання.

14. Яку роль виконує yield?

Повертає значення, призупиняє функцію та зберігає її стан для наступного `next()`.

15. Чим yield відрізняється від return?

`return` завершує функцію остаточно, а `yield` лише призупиняє її з можливістю продовження.

16. Коли фактично починає виконуватися generator function?

При першому `next()` (або першій ітерації), а не під час виклику функції.

17. Що зберігає generator між викликами next()?

Локальні змінні, позицію виконання та стан циклів і `with`-блоків.

18. Що таке generator expression?

Компактний lazy-вираз `(x for x in values if cond)`, який видає значення на запит.

19. Чим generator expression відрізняється від list comprehension?

List comprehension одразу будує список у пам’яті, generator expression не створює колекції й обчислює елементи лише за потреби.

20. Що таке lazy evaluation?

Обчислення елемента лише тоді, коли його запитує споживач.

21. Що таке eager evaluation?

Обчислення й збереження всіх результатів одразу, наприклад `list(...)` або list comprehension.

22. Які переваги lazy evaluation?

Менший peak memory, швидкий перший результат, early termination, робота з нескінченними потоками.

23. Які недоліки lazy evaluation?

Одноразовість, відсутність `len()` та індексації, відкладені помилки, довге життя ресурсів, складніше налагодження.

24. Що таке streaming?

Послідовна обробка даних частинами без повного завантаження набору в пам'ять.

25. Чому streaming важливий для великих files?

Пам'ять не залежить від розміру файлу, тому файл більший за RAM все одно можна обробити.

26. Чому read() може бути проблемою для multi-GB file?

`read()` завантажує весь вміст у RAM і може вичерпати пам'ять.

27. Чому file object є iterable?

Він реалізує iterator protocol і видає рядки по одному, тому підходить для `for` та `yield from`.

28. Для чого використовується yield from?

Делегує видачу значень іншому iterable/generator, замінюючи цикл `for item in x: yield item`.

29. Що таке itertools?

Стандартний модуль ефективних lazy-ітераторів (`chain`, `islice`, `count`, `groupby` та інші).

30. Для чого itertools.count()?

Створює нескінченну арифметичну послідовність із заданими початком і кроком.

31. Чому count() потрібно обмежувати?

Без обмеження потік нескінченний: `list(count())` ніколи не завершиться й вичерпає пам'ять; використовують `islice` або `takewhile`.

32. Для чого islice()?

Бере ділянку iterator без його повної матеріалізації (перші N, пакети).

33. Для чого chain()?

Об'єднує кілька iterable в один потік, не копіюючи дані.

34. Як працює takewhile()?

Видає елементи, доки умова істинна, і зупиняється на першому елементі, що її порушив.

35. Як працює dropwhile()?

Пропускає елементи, доки умова істинна, а потім повертає решту без перевірок.

36. Для чого accumulate()?

Видає накопичувальні результати (за замовчуванням суму): `[10, 30, 60]` для `[10, 20, 30]`.

37. Для чого pairwise()?

Повертає пари сусідніх елементів — зручно для різниць, змін показників та часових рядів.

38. Як працює groupby()?

Групує **послідовні** елементи з однаковим ключем і видає пари «ключ — група».

39. Чому перед groupby() часто потрібне sorting?

Бо групуються лише сусідні елементи; сортування ставить однакові ключі поруч.

40. Що робить product()?

Генерує декартів добуток кількох iterable.

41. Чим combinations() відрізняється від permutations()?

`combinations` не враховує порядок елементів у вибірці, `permutations` — враховує.

42. Чому combinatorial iterators можуть бути небезпечними навіть при lazy execution?

Кількість результатів зростає factorial/експоненційно, тому навіть lazy-перебір може тривати вічно.

43. Що таке batching?

Розбиття потоку на пакети обмеженого розміру (`batched()`).

44. Які переваги batching?

Обмежений обсяг пам'яті (O(b)) та ефективніші операції з API, БД і файлами, що працюють блоками.

45. Що таке lazy pipeline?

Ланцюг generator-етапів, у якому записи проходять поступово без повних проміжних списків.

46. Які stages може містити pipeline?

Source, parsing, validation, filtering, transformation, aggregation, batching та sink.

47. Що є source у streaming pipeline?

Джерело даних: файл `cars.csv`, кілька файлів, база даних чи інший генератор.

48. Що є transformation?

Етап, що перетворює запис в інший вигляд (`summarize_cars()`, `format_car_rows()`).

49. Що є sink?

Кінцевий споживач потоку: агрегація, запис у файл/БД або вивід (`calculate_statistics()`).

50. Чи гарантує generator менший total execution time?

Ні: у виміряному експерименті повний час був зіставним; generator економить насамперед пам’ять.

51. Яка основна memory advantage generator?

Зберігається лише поточний елемент і стан, а не вся колекція — пам’ять практично не залежить від обсягу даних.

52. Що таке time to first result?

Час від старту обробки до появи першого результату: у lazy-варіанті він майже сталий.

53. Чому lazy pipeline добре підтримує early termination?

Споживач може припинити запит наступних елементів, тому решта даних не читається (`find_car_by_id`, `first_matching_cars`).

54. Чому exception у generator може виникати із затримкою?

Тіло generator виконується під час споживання, тому помилка з’являється лише при запиті відповідного елемента.

55. Чому list(generator) може бути проблемою?

Він матеріалізує всі значення й нівелює перевагу пам’яті; для великих потоків може вичерпати RAM.

56. Що трапиться при list(count())?

Програма не завершиться нормально: список росте нескінченно, доки не вичерпається пам’ять.

57. Як generator може вплинути на lifecycle file resource?

`with` у generator лишається відкритим, поки його не вичерпано або не закрито `close()`; незавершене споживання тримає файл відкритим.

58. Як тестувати невеликий generator?

Перетворити результат на `list()` і порівняти з очікуваним; перевірити крайні випадки (порожній вхід, помилкові рядки), як у `test_streaming.py`.

59. Як тестувати infinite generator?

Обмежити його `islice` або кількома `next()` і перевірити префікс (`first_car_ids(5, start=100)`).

60. Коли краще використовувати list, а коли generator?

List — для малих даних, індексації та повторних обходів; generator — для великих, потокових даних, одноразового обходу та раннього завершення.

31. Висновок

У лабораторній роботі №3 розроблено потоковий конвеєр обробки каталогу автомобілів (варіант №5) у підпакеті `car_catalog.streaming`. CSV-

файл проходить через етапи читання, очищення, розбору, валідації, фільтрації, перетворення та агрегації; усі етапи реалізовано як generator functions, тому дані обробляються запис за записом без повного завантаження файлу в пам'ять.

Реалізовано власний iterator із протоколом `__iter__()` / `__next__()`, generator functions з `yield`, `yield from` (у тому числі рекурсивний), generator expressions та шість засобів `itertools` (`chain`, `islice`, `count`, `accumulate`, `pairwise`, `groupby`), пакетну обробку, облік некоректних записів і пошук із достроковим завершенням.

Експеримент на 10 000, 100 000 та 500 000 записів показав, що пікова пам'ять eager-обробки зростає лінійно (до 266.883 MiB), а lazy-обробки лишається практично сталою (≈0.055 MiB); час до першого результату в lazy-варіанті також майже не залежить від розміру. Повний час обох підходів виявився зіставним, тож перевага lazy-підходу — насамперед у пам'яті, швидкому першому результаті та early termination: пошук першого запису серед мільйона виконано за 0.0002 с проти 3.83 с в eager-варіанті. Водночас lazy-обробка має обмеження — одноразовість потоку та відкладені помилки; операції на кшталт сортування для `groupby` потребують матеріалізації. Код покрито 40 автоматичними тестами `pytest`, усі вони успішні.